Chapter 3

鏈結串列結構

3.1 前言

- 陣列結構 (Array) 和鏈結串列 (Linked List) :
  - 陣列的使用是透過陣列變數及對應的索引來存取陣列個別元素。
  - 鏈結串列儲存資料的方式是將個別資料項次 (data item) 透過鏈結串在一起。

3.2 單向鏈結串列 (Singly Linked List)

- 給定一組未經排序的六筆資料 {5 ， 78 ， 65 ， 31 ， 83 ， 98} ，經由小而大排序後再存入一個大小為 10 的一維陣列 DATA 中，如圖 3.1 所示。

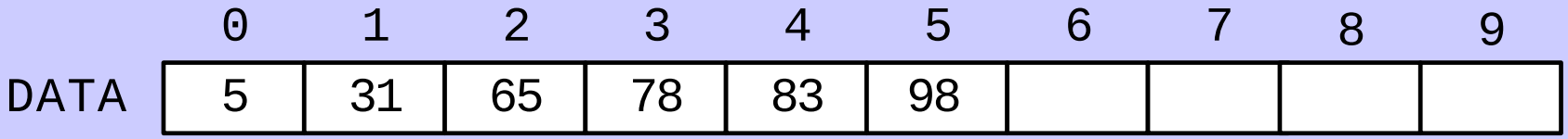


圖 3.1　陣列結構 DATA[10]

假設我們希望將資料 45 加入這些排序過的資料中並維持由小到大的順序，利用陣列結構在處理資料的插入是相當麻煩的。

3.2 單向鏈結串列 (Singly Linked List)

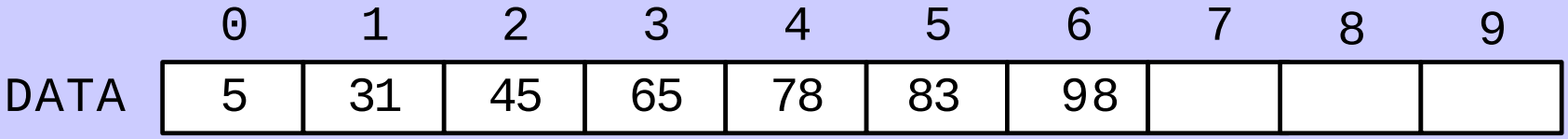


圖 3.2　陣列結構 DATA[10]（插入 45 後的結果）

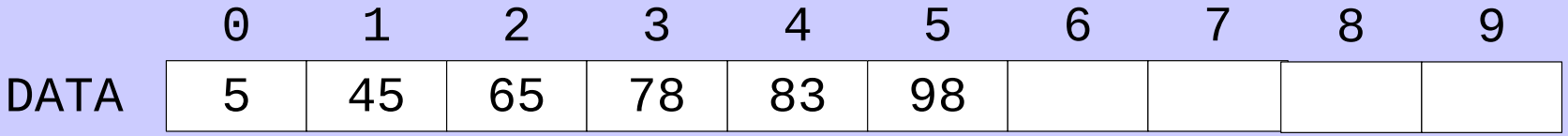


圖 3.3　陣列結構 DATA[10]（刪除 31 後的結果）

3.2 單向鏈結串列 (Singly Linked List)

- 以 C 語言為例，宣告單向鏈結串列的語法如下：

【範例】定義一個單向鏈結串列之節點資料型態

```
struct node{                    /*  定義一個單向鏈結節點類別（結構）  */
   int            data;         /* data  用來儲存節點資料值  */
   struct node    *link;        /*  為一個  node  指標，它指向下一個節點  */
};
```

3.2 單向鏈結串列 (Singly Linked List)

- 我們將利用圖 3.7 中一個包含六筆資料的有序單向鏈結串列為例子來介紹幾種基本的運作。

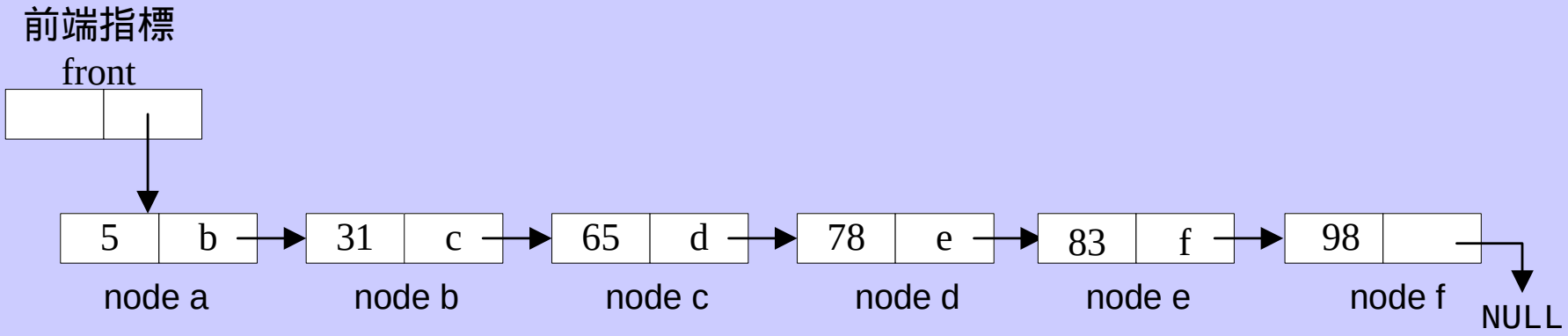


圖 3.7　有序單向鏈結串列

3.2 單向鏈結串列 (Singly Linked List)

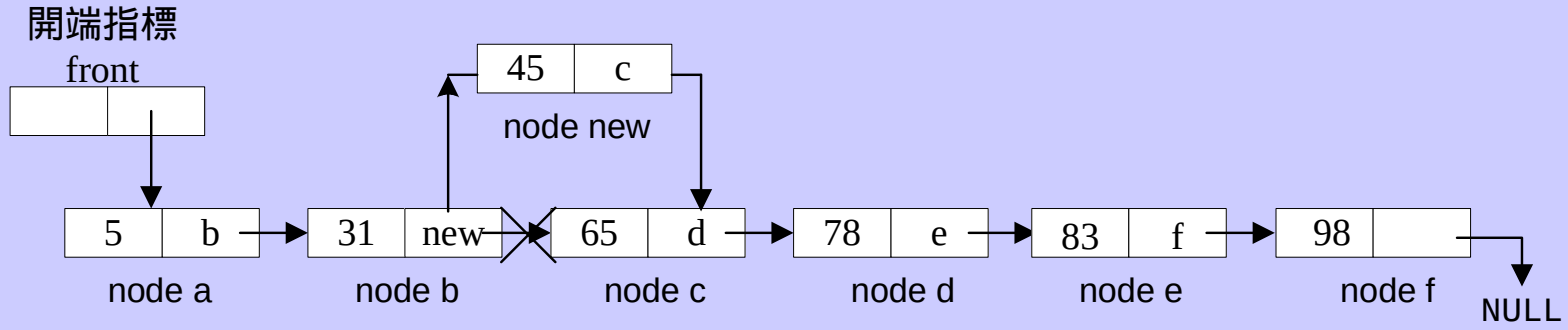


圖 3.8　將資料 45 插入圖 3.6 之鏈結串列之結果

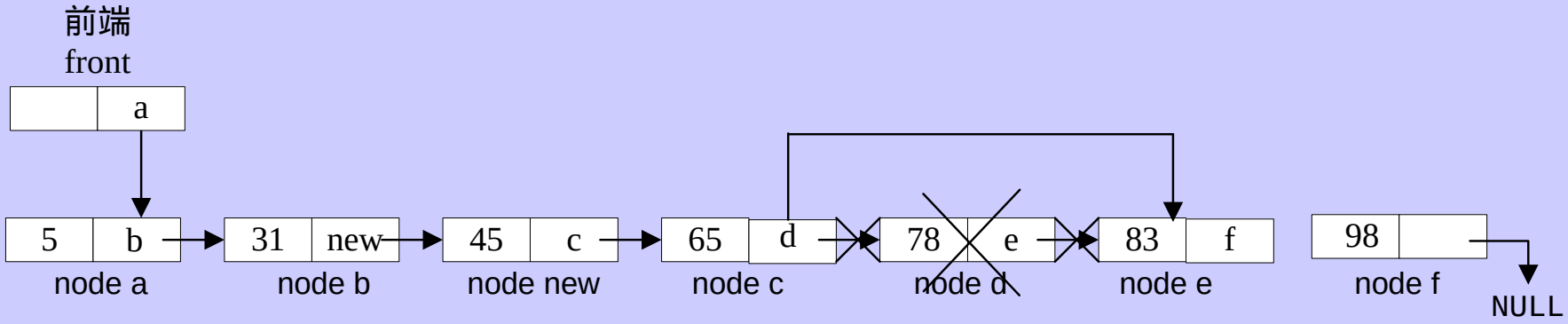


圖 3.9　從鏈結串列中刪除資料 78

3.2.1 堆疊的應用

- 堆疊處理資料具有「先進後出」(First In Last Out, FILO) 的特性。

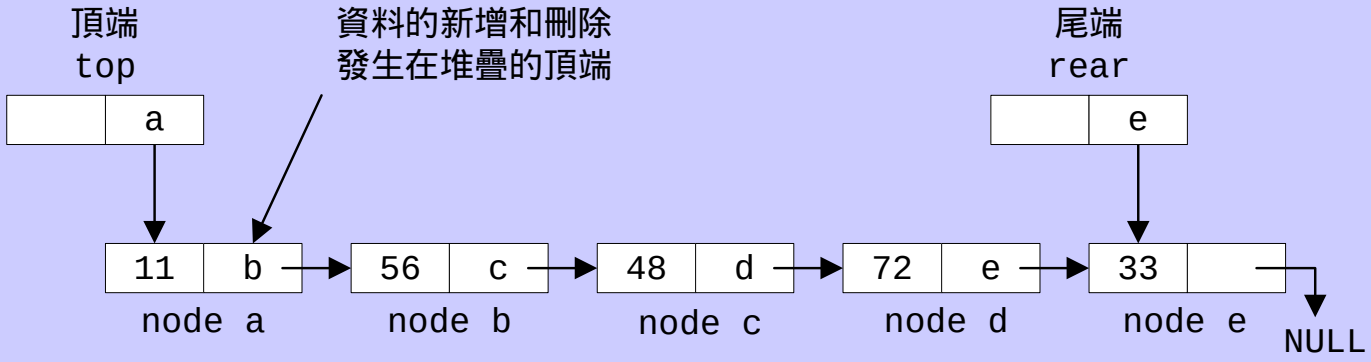


圖 3.10 用鏈結串列表示的堆疊結構

3.2.2 佇列的應用

- 佇列處理資料具有「先進先出」(First In First Out, FIFO)的特性。

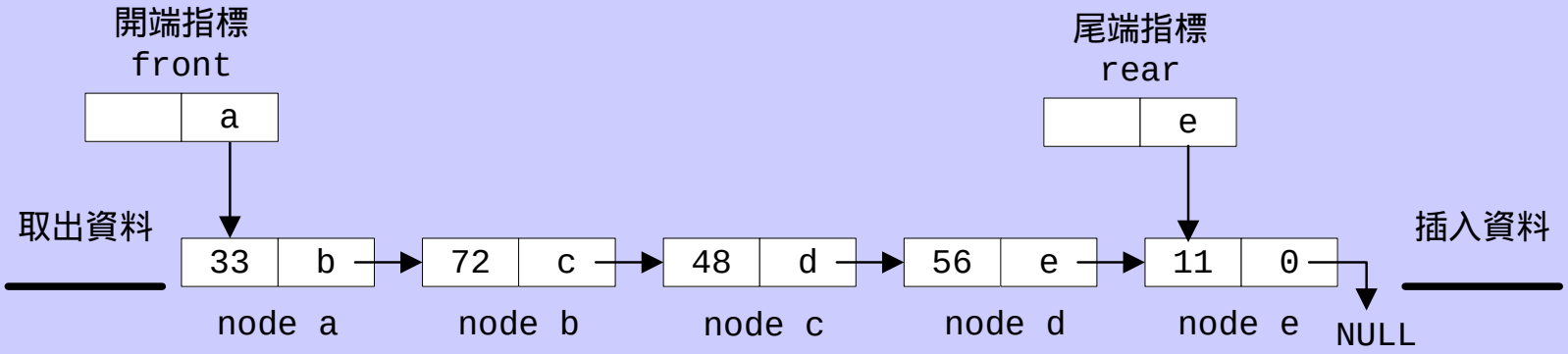


圖 3.11 用單向鏈結串列表示的佇列結構

3.2.2 佇列的應用

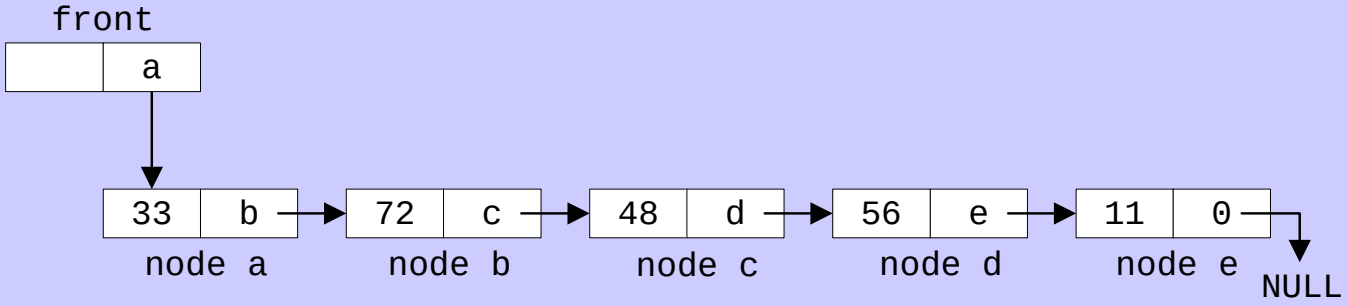
- 以 C 語言為例，產生一個空佇列的語法如下：

【產生一個空佇列】

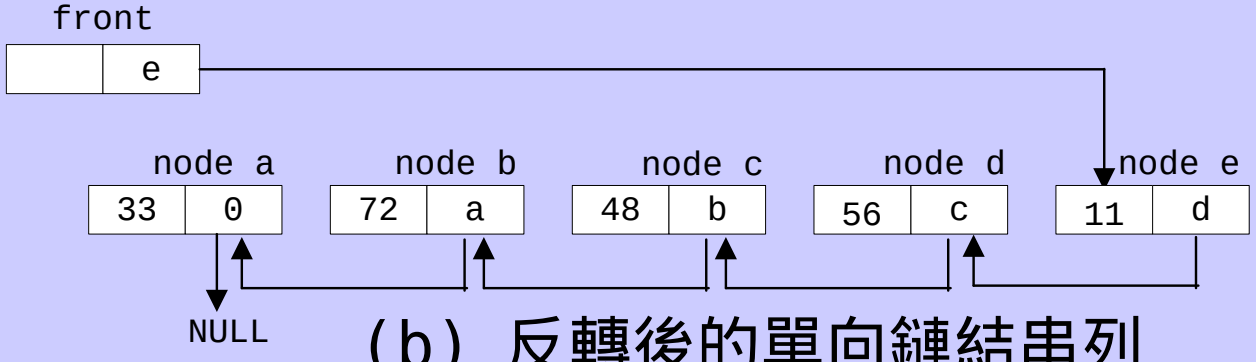
```
void create_empty_queue(void)
{
   struct node *front,*rear;
   front = (struct node *) malloc(sizeof(struct node));
   rear = (struct node *) malloc(sizeof(struct node));
   front->link = NULL;
   rear->link = NULL;
}
```

3.2.3 將單向鏈結串列反轉

- 單向鏈結串列是一個具有方向性的資料表示法，要將一個單向鏈結串列反轉就是要將節點與節點之間的順序關係倒轉。



(a) 反轉前的單向鏈結串列



(b) 反轉後的單向鏈結串列

圖 3.12 單向鏈結串列之反轉範例

3.2.3 將單向鏈結串列反轉

- 以 C 語言為例，反轉一個單向鏈結串列的處理程序列語法如下：

【反轉一個單向鏈結串列】

```
int invert_list(struct node *head)
{
   struct node *mid_node, *last_node;
   mid_node=NULL;
   while( head!=NULL )
      {
         last_node=mid_node;                          /*  把中間節點傳遞給結尾指標  */
         mid_node=head;                               /*   把前頭節點傳遞給中間指標  */
         head=head->link;                               /*   前頭指標往前進一個節點  */
         mid_node->link=last_node;       /*  反轉原本的前頭與中間節點順序關係  */
      }
  return(mid_node);                                     /*  將反轉後的開端指標傳回  */
}
```

3.2.4 計算串列之長度

- 串列之長度指的是串列中儲存資料之節點個數，不包含 front 及 rear。
- 以 C 語言為例，計算串列之長度語法如下：

【計算串列之長度】

```
void length(struct node *front_)
{
   int count = 0;
   struct node *this_node;
   if (front_->link != NULL)                    /* queue is not empty */
      {
        this_node = front_->link;
        while(this_node->link != NULL)
             {
               count ++;
```

3.2.4 計算串列之長度

```
                this_node = this_node->link;
            }
        count ++;
        printf(" ==>  串列長度為  : %d\n",count);
    }
  else
        printf(" !!!  空串列，串列長度為  : 0 \n");
}
```

3.2.5 串列之合併

- 假設我們希望將兩個單向鏈結串列 x 與 y 合併，我們必須將串列 y 中的所有節點合併到串列 x 的所有節點後面，產生一個合併後的串列 z。在合併的處理中我們發現可能會有下列三種可能狀況出現：

(1). 當串列 x 是空的則合併後的串列 z 就是串列 y。

(2). 當串列 y 是空的則合併後的串列 z 就是串列 x。

(3). 如果串列 x 與 y 都不是空串列，則必須將串列 y 中所有的節點搬移到串列 x 的所有節點之後，即可完成兩個串列合併的動作。

3.2.5 串列之合併

- 假設串列 x 與 y 皆為非空的串列，我們發現合併的動作包含下列幾個步驟：

1. 將 front_z 指向串列 x 的第一個節點。
2. 找到 x 串列的最後一個節點。然後設定此節點的鏈結欄位指向串列 y 的第一個節點。

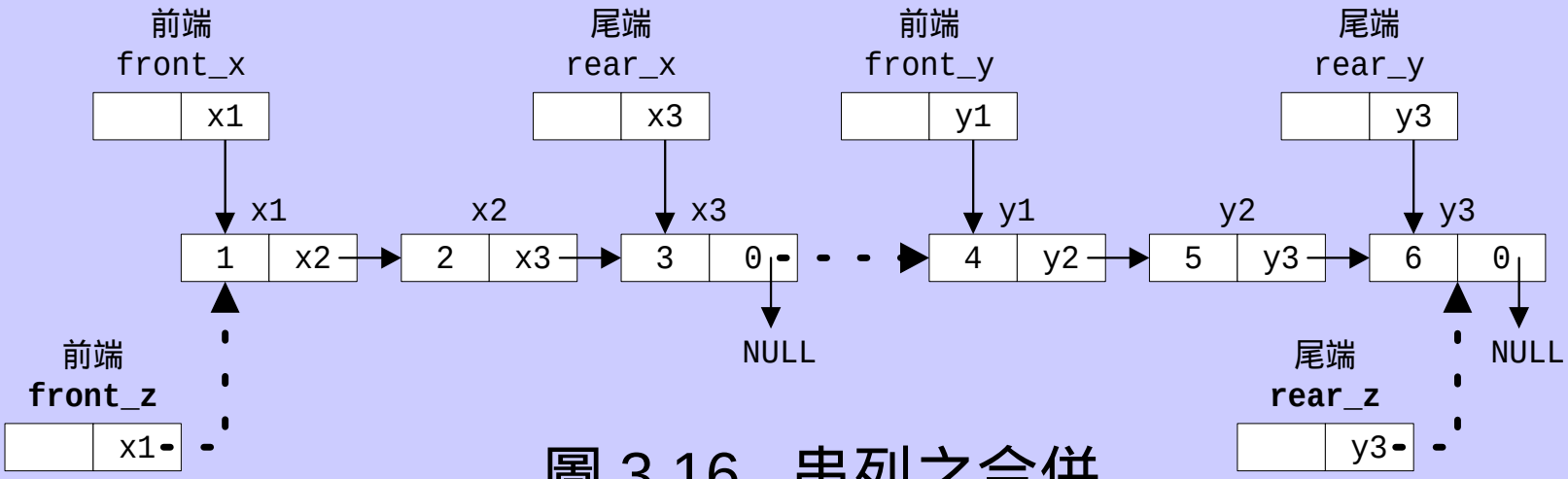


圖 3.16　串列之合併

3.2.5 串列之合併

- 以 C 語言為例，將 y 串列合併到 x 串列之後，並以 front_z 為新串列的前端，其語法如下：

【將 y 串列合併到 x 串列之後，並以 front_z 為新串列的前端】

```
void cancatenate(struct node *front_x, struct node *front_y,
                                       struct node *front_z)
{
   struct node *this_node;
   if (front_x->link == NULL)                    /* front_x 為空串列 */
        front_z->link = front_y->link;
   else
     {
        if (front_y->link == NULL)               /* front_y 為空串列 */
           {
             front_z->link = front_x->link;
           }
```

3.2.5 串列之合併

```
            else                          /* front_x, front_y 均非空串列 */
          {
            front_z->link = front_x->link;
            this_node = front_x->link;
            while(this_node->link != NULL)
               this_node = this_node->link;
            this_node->link = front_y->link;
          }
      }
}
```

3.2.6 釋放鏈結串列的空間

- 當我們在程式的執行過程中不再需要用到某個鏈結串列時，我們便需要將此鏈結串列所使用的空間加以釋放。詳細的 C 語言程式片斷如下所列：

【歸還單向鏈結串列】

```
void free_list(struct node *front_)
{
  struct node *this_node;
  struct node *temp_node;

  if (front_->link != NULL)                    /* queue is not empty */
      {
         this_node=front_link;
         while ( this_node != NULL )
           {
             temp_node=this_node;
```

3.2.6 釋放鏈結串列的空間

```
                this_node=this_node->link;
              free(temp_node);
            }
        }
    else
        {
            printf("Find a empty list.\n");
        }
}
```

3.3 雙向鏈結串列

- 雙向鏈結串列包含了許多個節點以及紀錄節點間順序關係的鏈結。
- 一個雙向鏈結串列的節點包含三個欄位：

(1). 左鏈結欄位 (Left Link Field, LLINK) :

LLINK 鏈結欄是用來指向前一個節點。

(2). 資料欄位 (Data Filed) :

data 資料欄仍然用來儲存資料。

(3). 右鏈結欄位 (Right Link Field, RLINK) :

RLINK 欄則用來指向下一個節點。

3.3 雙向鏈結串列



圖 3.17　雙向鏈結串列的節點結構

- 雙向鏈結串列的一個節點比起單向鏈結串列的一個節點需要更多的記憶體空間，因為雙向鏈結串列的節點有兩個鏈結欄位而單向鏈結串列的節點只有一個鏈結欄位。

- 雙向鏈結串列的優點為資料較不容易遺失；相對於單向鏈結串列或環狀串列萬一不幸其中有一個鏈結斷裂，那麼後面的串列資料便遺失而永遠無法再復原了。

3.3 雙向鏈結串列

- 以 C 語言為例，定義一個雙向鏈結串列之節點資料型態語法如下：

【範例】定義一個雙向鏈結串列之節點資料型態

```
struct node{                                    /*  定義一個單向鏈結節點類別（結構）  */
   int                data;                    /* data  用來儲存節點資料值  */
   struct node  *l_link;                       /*  為一個  node  指標，它指向前一個節點
*/
   struct  node *r_link;                       /*  為一個  node  指標，它指向下一個節點  *
/
};
```

3.3 雙向鏈結串列

【範例】利用雙向鏈結串列結構來表示一組由小到大排序的六筆資料{5 ， 31 ，65 ， 78 ，83 ，98}。

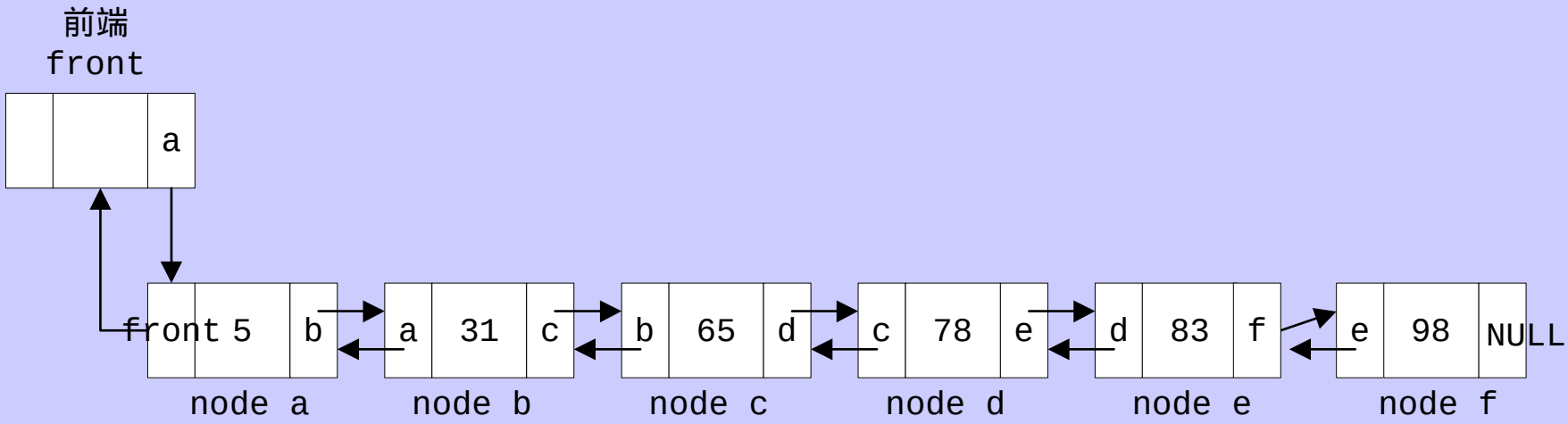


圖 3.18　只有使用開端節點指標的雙向鏈結串列

3.3 雙向鏈結串列

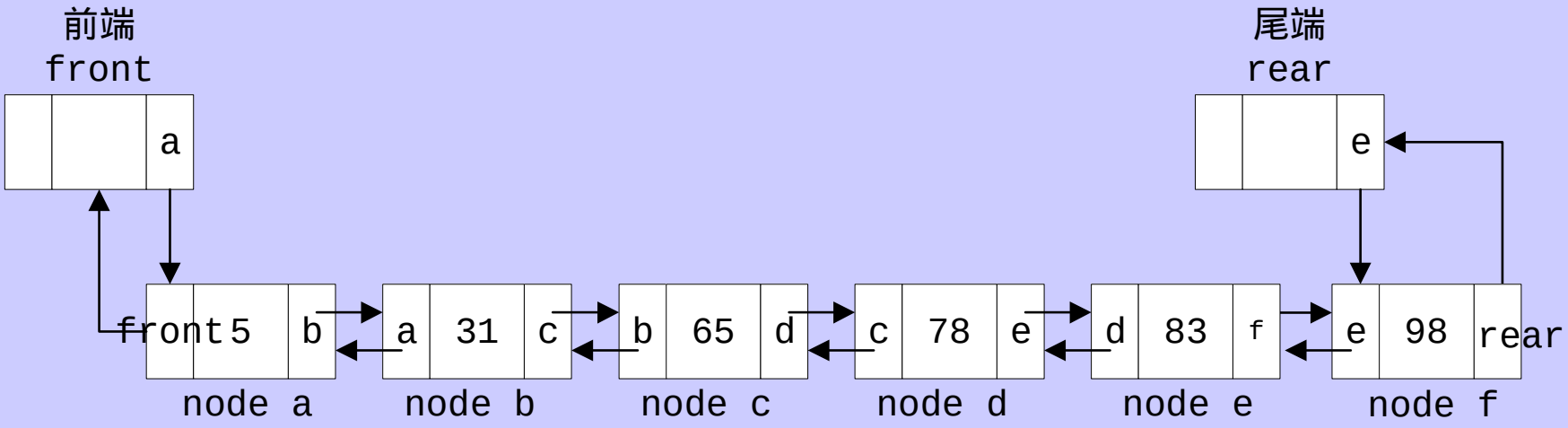


圖 3.19　使用開端與節尾節點指標的雙向鏈結串列

3.3 雙向鏈結串列

【範例】插入資料 45 到圖 3.18 的雙向鏈結串列之詳細過程列於圖 3.20 。

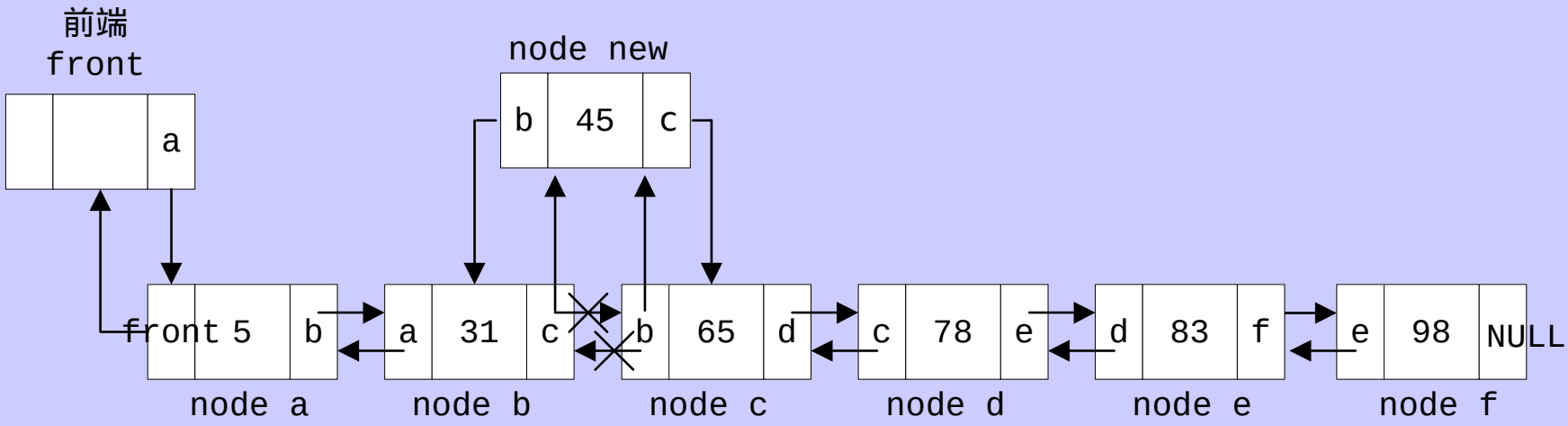


圖 3.20　從雙向鏈結串列插入資料 45 的鏈結改變示意圖

3.3 雙向鏈結串列

【範例】從圖 3.18 的雙向鏈結串列刪除資料 78 。

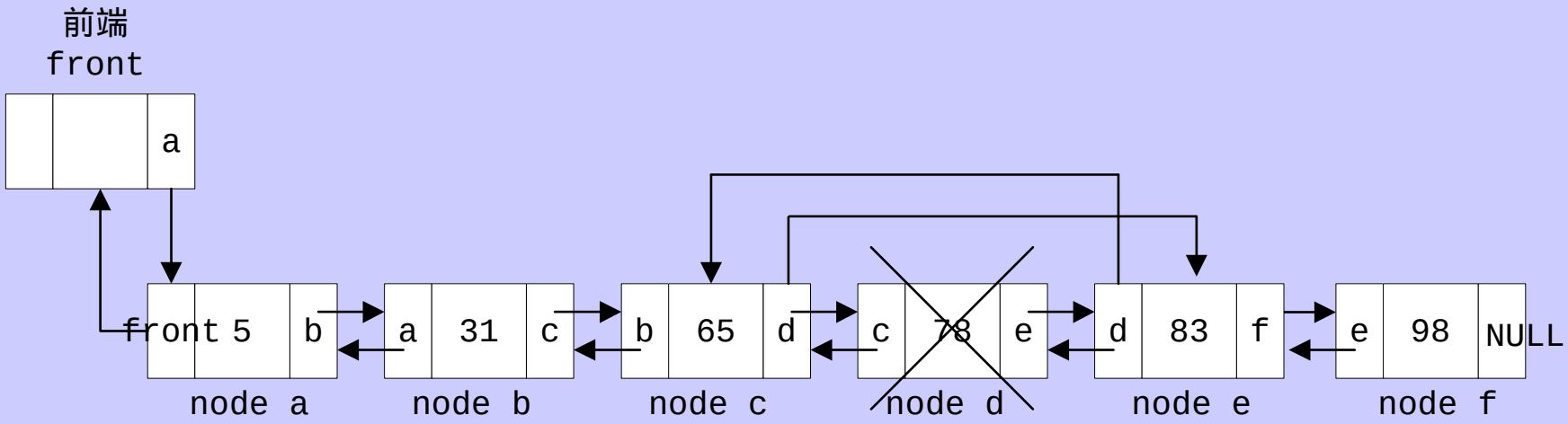


圖 3.21　從圖 3.18 的雙向鏈結串列刪除資料 78 之鏈結改變示意圖

3.4 環狀串列 (Circular Linked List)

- 環狀串列 (Circular Linked List) 是鏈結串列結構中常見的一種變形結構。

- 在 3.2 節所介紹的單向鏈結串列與 3.3 節所介紹的雙向鏈結串列在表示最後一個節點時將單向鏈結串列的節點鏈結欄位 (Link) 與雙向鏈結串列的右鏈結欄位 (RLINK) 設定是空的 (NULL)。這樣的做法若是串列的開端指標受到破壞就會造成整個串列的資料流失。

3.4.1 環狀單向鏈結串列

- 環狀單向鏈結串列：

將單向鏈結串列(Singly Linked List)的最後一個節點的指標欄位指向串列的第一個節點就會形成一個具有單一方向性的環型結構，也就是說鏈結單向串列(Circular Singly Linked List)的最後一個節點之鏈結欄不再指向NULL(0或接地)而是指向鏈結串列的第一個節點。

3.4.1 環狀單向鏈結串列

【範例】將圖 3.7 中一個包含六筆資料的有序單向鏈結串列轉換成單向環狀鏈結串列。

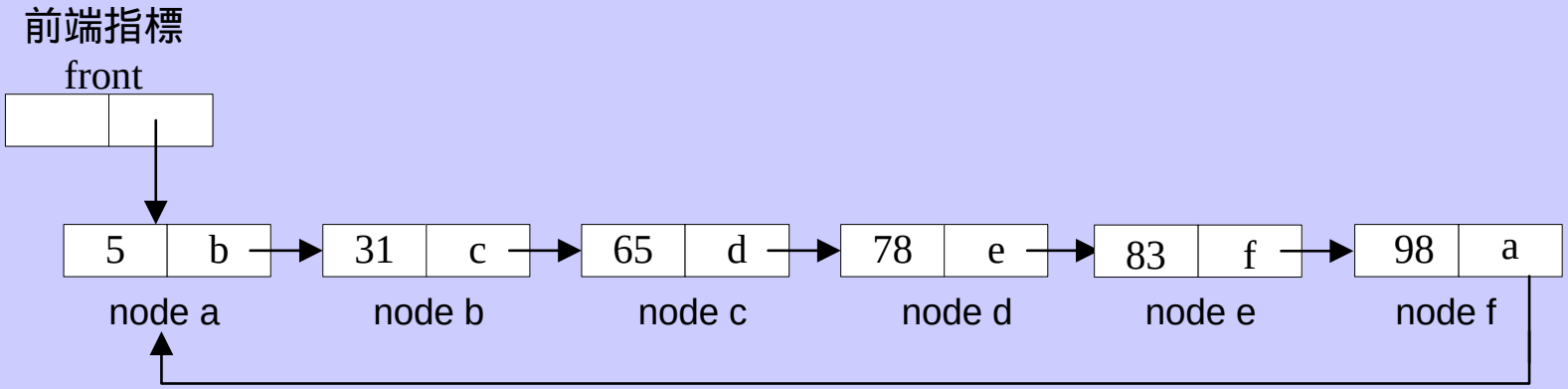


圖 3.22　環狀單向鏈結串列

3.4.1 環狀單向鏈結串列

【範例】新增資料 2 到圖 3.22 中的環狀單向鏈結串列。

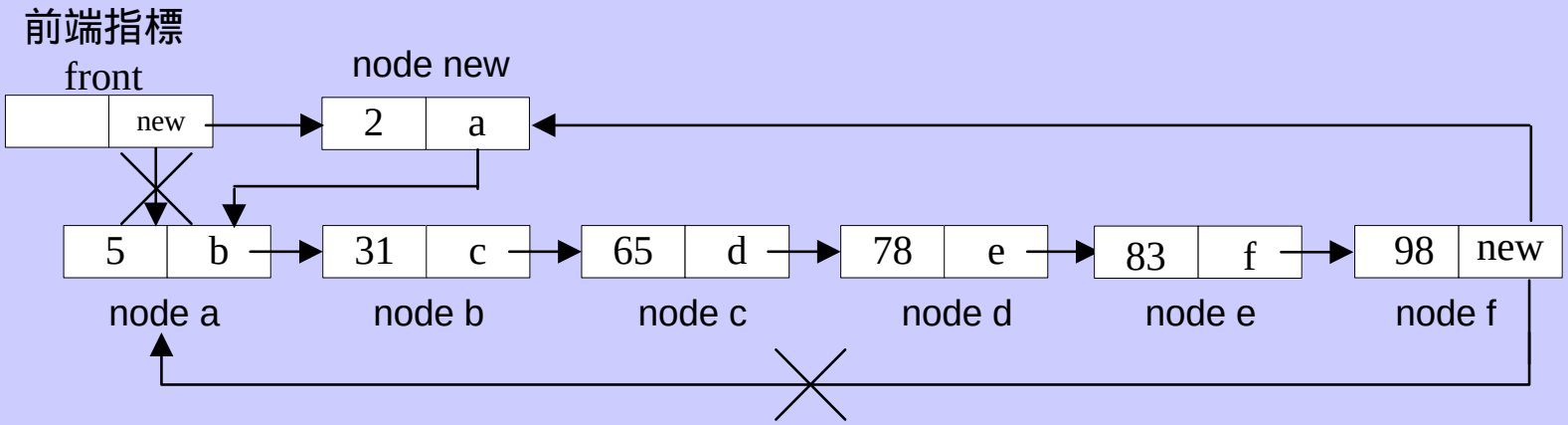


圖 3.23　新增資料到環狀單向鏈結串列的第一個節點

3.4.1 環狀單向鏈結串列

【範例】刪除圖 3.22 的環狀單向鏈結串列第一個節點。

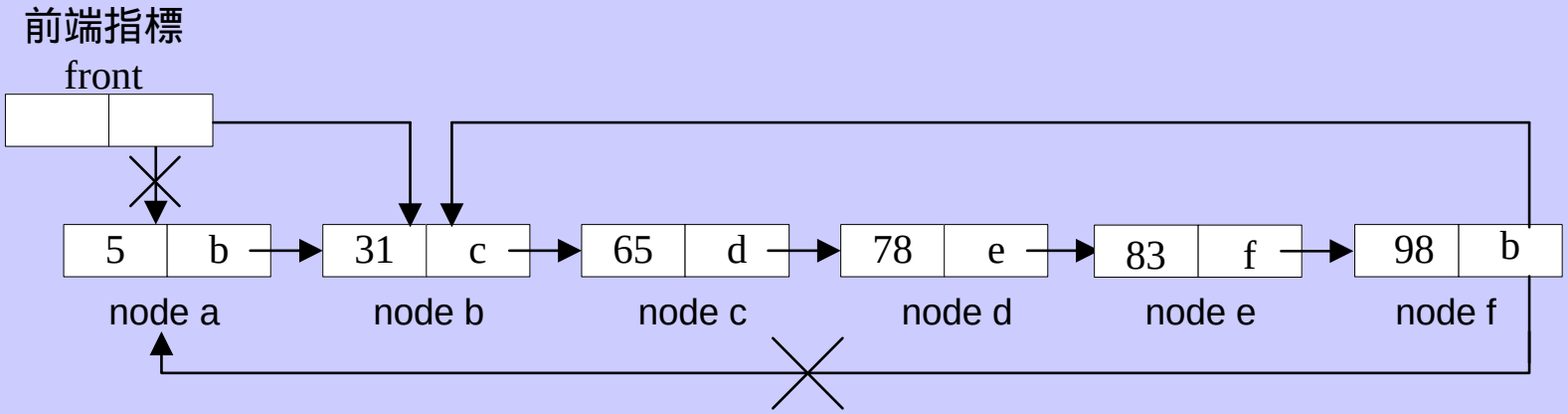


圖 3.24　刪除環狀單向鏈結串列的第一個節點

3.4.2 環狀雙向鏈結串列

【範例】利用圖 3.18 中一個包含六筆資料的有序雙向鏈結串列轉換成環狀雙向鏈結串列。

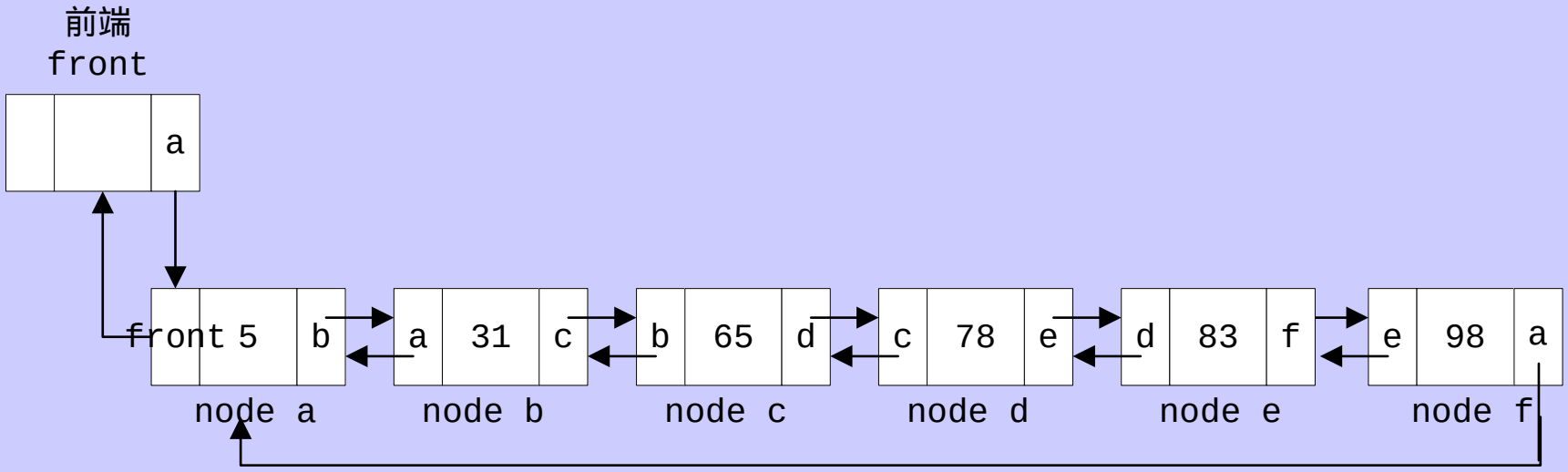


圖 3.25　環狀雙向鏈結串列

3.4.3 利用環狀鏈結串列表示稀疏矩陣

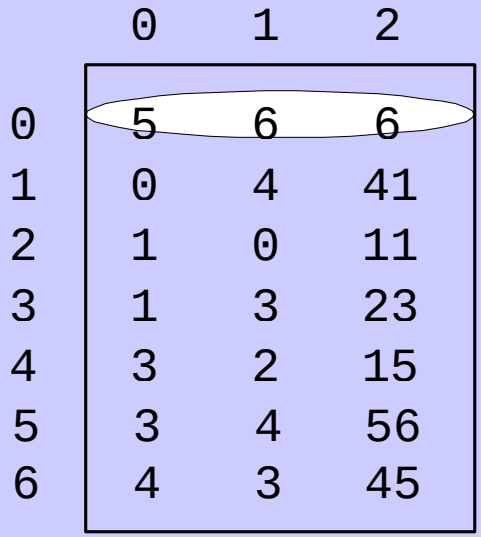
- 稀疏矩陣 (Sparse Matrix):

當一個矩陣的非零元素之個數遠低於零元素之個數，我們就稱該矩陣為稀疏矩陣。

	0	1	2	3	4	5
0	0	0	0	0	41	0
1	11	0	0	23	0	0
2	0	0	0	0	0	0
3	0	0	15	0	56	0
4	0	0	0	45	0	0

圖 3.26 5×6 稀疏矩陣

3.4.3 利用環狀鏈結串列表示稀疏矩陣



	0	1	2
0	5	6	6
1	0	4	41
2	1	0	11
3	1	3	23
4	3	2	15
5	3	4	56
6	4	3	45

圖 3.27　用 7*3 矩陣表示圖 3.26

3.4.3 利用環狀鏈結串列表示稀疏矩陣

【範例】利用環狀串列來表示稀疏矩陣的內容。

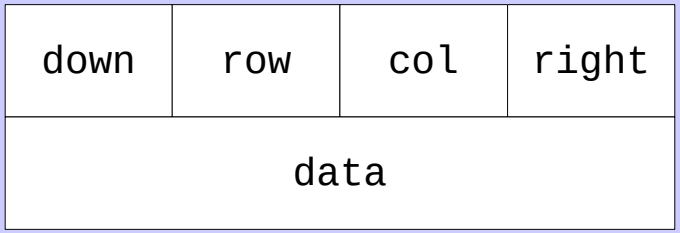


圖 3.28　用以表示二維稀疏矩陣的節點結構

3.4.3 利用環狀鏈結串列表示稀疏矩陣

- 利用 C 語言來描述環狀串列節點的資料型態宣告列於下：

【範例】定義一個利用環狀串列儲存稀疏矩陣的節點資料型態

```
struct node{                          /*  定義一個環狀鏈結節點類別（結構）  */
   int              row;             /* row  用來儲存非零元素的列數  */
   int              col;             /* col  用來儲存非零元素的行數  */
   int              data;            /* data 用來儲存非零元素的數值  */
   struct node   *right;             /*  為一個  node  指標，它指向同一列的下一個節點  *
/
   struct  node   *down;             /*  為一個  node  指標，它指向同一行的下一個節點  *
/
};
```

3.4.3 利用環狀鏈結串列表示稀疏矩陣

【範例】稀疏矩陣的環狀串列表示法。

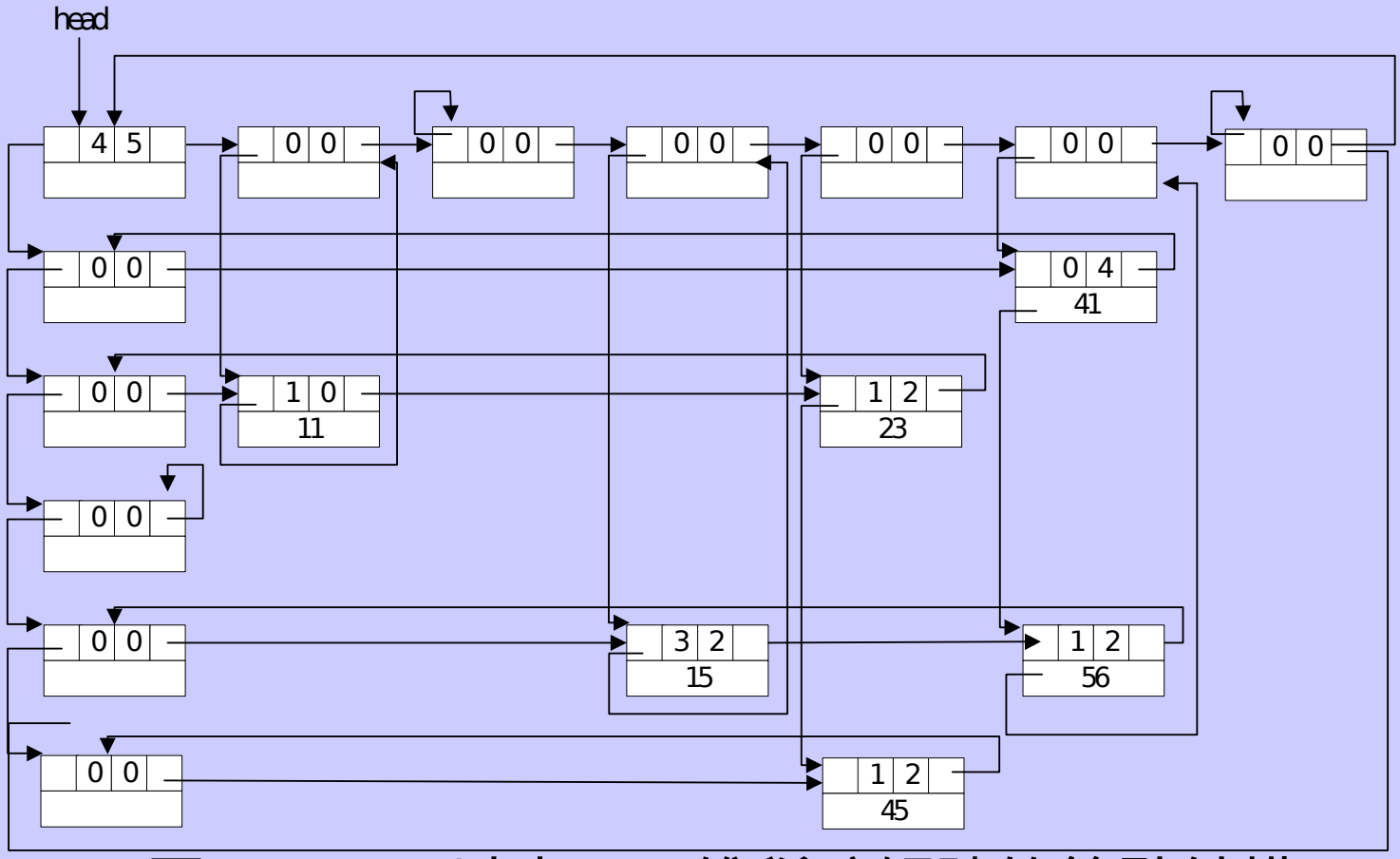


圖 3.28　用以表示二維稀疏矩陣的節點結構

3.5 多項式表示法

- 多項式：

$f(x)=a_nx^n+a_{n-1}x^{n-1}+\ldots+a_1x^1+a_0x^0$ 是一個單變數 x 之多項式，其中 a_n ， a_{n-1} ，… ， a_1 ， a_0 為此多項式之係數 (Coefficient) ，而 n ， n-1 ，… ， 1 ， 0 等為多項式的指數 (Exponent) 次方之數值。

【範例】多項式 $f(x)=6x^5+12x^3+11$ 。

	0	1	2	3	4	5	6
DATA	5	6	0	12	0	0	11
	n	x^5	x^4	x^3	x^2	x^1	x^0

圖 3.30 利用陣列來表示給定的單變數多項式

3.5 多項式表示法

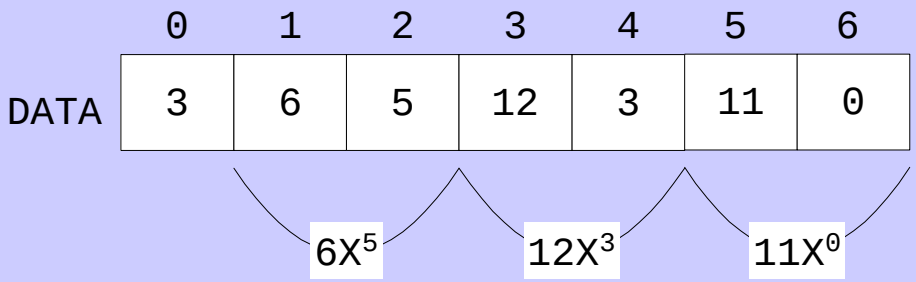


圖 3.31　另外一個利用陣列來表示給定的單變數多項式

3.5 多項式表示法

- 利用鏈結串列來表示單一變數多項式。



圖 3.32　用來表示單變數多項式非零項次的節點結構

【範例】利用單向鏈結串列表示多項式

$f(x)=6x^5+12x^3+11$　。

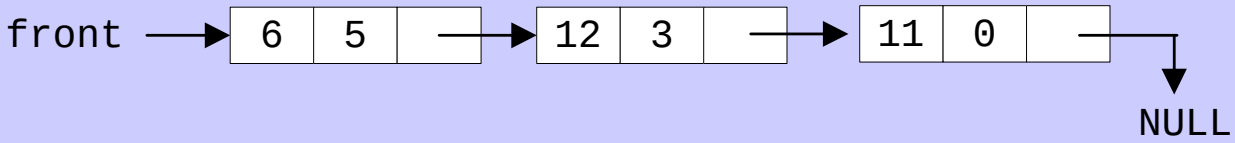


圖 3.33　利用單向鏈結串列表示多項式 $f(x)=6x^5+12x^3+11$

3.5 多項式表示法

- 利用鏈結串列來表示多變數多項式。

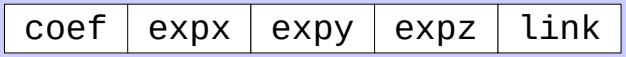


圖 3.34 用來表示多變數多項式非零項次的節點結構

【範例】利用單向鏈結串列表示多變數多項式

$f(x,y,z)=6x^5y^4z^3+4x^3y^2z+3xy$ 。

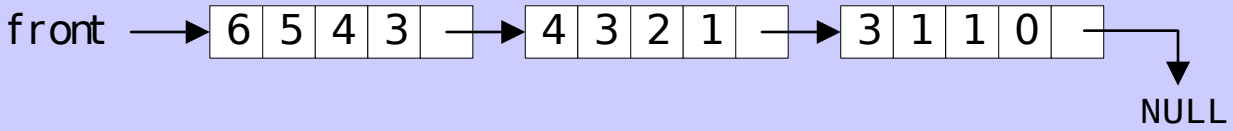


圖 3.35 利用單向鏈結串列表示多項式 $f(x,y,z)= 6x^5y^4z^3+4x^3y^2z+3xy$